S2S-04: Step 4 — Prepare: Export and Pre-Stage

> **Series:** S2S | **Notebook:** 4 of 10 | **Phase:** Upgrade | **Step:** Prepare | **Created:** March 2026 | **Last Updated:** 04/16/2026

Overview

With the target tenant architecture designed (Step 3), this step builds the foundation. Preparation is the longest step in any SaaS-to-SaaS migration — it involves exporting configuration from the source tenant, provisioning the target tenant, deploying IAM, installing ActiveGates, preparing Kubernetes operators, and freezing the source tenant to prevent drift during execution.

Every task in this notebook must be completed and validated before Step 5 (Execute) begins. Skipping preparation tasks is the primary cause of migration rollbacks.

S2S Migration Framework

Phase	Steps	Focus
Plan	1. Discover → 2. Strategize → 3. Design	Understand what exists, choose approach, design target
Upgrade	**4. Prepare** → 5. Execute → 6. Integrate	Build target, move config, connect integrations
Run	7. Expand → 8. Enable → 9. Optimize	Roll out agents, enable teams, tune and decommission

> **You are here: Step 4 — Prepare.** You have designed the target architecture (Step 3). Now you export configuration, provision the target tenant, and stage all prerequisites before execution begins.

Table of Contents

Prerequisites

Requirement	Details
Step 3 Complete	Target tenant architecture designed — IAM, buckets, OpenPipeline, deployment order documented
Source Tenant Access	Admin access with `ReadConfig`, `settings.read`, `activeGates.read` scopes
Target Tenant Provisioned	Dynatrace SaaS environment with Grail enabled
Monaco CLI	`monaco v2.x` installed and configured
Terraform	`terraform >= 1.0` with Dynatrace provider `~> 1.91`
Helm	`helm >= 3.x` for Kubernetes operator deployment
Change Window	Approved change window for configuration freeze

Order of Operations

This notebook covers **operations 1–3** of the Upgrade phase:

#	Operation	Section	Dependency
1	**Provision target tenant and configure SSO/IAM**	Sections 1–2	Step 3 design deliverables
2	**Export configuration from source tenant**	Section 3	Source tenant access
3	**Deploy ActiveGates and prepare K8s operators**	Sections 4–5	Target tenant provisioned
4	Import configuration to target tenant	Step 5	Operations 1–3 complete
5	Redirect agents to target tenant	Step 5	Configuration imported
6	Validate data flow in target tenant	Step 5	Agents redirected
7	Reconnect cloud integrations	Step 6	Agents reporting
8	Migrate dashboards, workflows, and extensions	Step 6	Integrations connected

> **Bold** operations are covered in this notebook. Operations 4–8 are covered in Steps 5 and 6.

1. Target Tenant Provisioning

The target tenant must be provisioned and configured with baseline settings before any configuration is imported.

Provisioning Checklist

Item	Action	Status
Region selection	Choose region closest to primary workload (same region as source if relocation)	☐
Environment activation	Activate SaaS environment via Dynatrace Account Management	☐

Item	Action	Status
Grail enablement	Verify Grail is enabled (default for new SaaS tenants)	☐
API token creation	Create token with `WriteConfig`, `ReadConfig`, `settings.write`, `settings.read` scopes	☐
OAuth client creation	Create OAuth client for account-level API access (IAM management)	☐
Cluster version	Verify target tenant is on same or newer cluster version than source	☐

Region Considerations

Scenario	Region Strategy
Relocation	Same region as source (minimize latency change)
Consolidation	Region closest to majority of monitored infrastructure
Cloud transformation	Region matching the target cloud provider's primary region
Compliance-driven	Region that satisfies data residency requirements (EU, US, APAC)

Pre-Migration Baseline: Entity Counts

Before any migration begins, capture entity counts from the source tenant. These serve as validation targets after agent cutover in Step 5.

```
// Source tenant: baseline host count
fetch dt.entity.host
| summarize host_count = count()
| fieldsAdd entity_type = "Hosts"

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes HOST
// | summarize host_count = count()
// | fieldsAdd entity_type = "Hosts"
```

```
// Source tenant: baseline service count
fetch dt.entity.service
| summarize service_count = count()
| fieldsAdd entity_type = "Services"

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes SERVICE
// | summarize service_count = count()
// | fieldsAdd entity_type = "Services"
```

```
// Source tenant: baseline process group count
fetch dt.entity.process_group
| summarize pg_count = count()
| fieldsAdd entity_type = "Process Groups"

// Alternative: Smartscape on Grail (entity.name → name)
// smartscapeNodes PROCESS
// | summarize pg_count = count()
// | fieldsAdd entity_type = "Process Groups"
```

```
// Source tenant: Grail bucket inventory for migration planning
fetch dt.system.buckets
| fields display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| fieldsAdd size_gb = estimated_uncompressed_bytes / 1073741824.0
| sort size_gb desc
```

> **Record these counts.** After agent cutover in Step 5, you will re-run these queries against the target tenant and compare. Entity counts within 5% indicate a successful migration. Counts below 90% indicate missing agents or misconfigured host groups.

2. SSO and IAM Setup

IAM is the first configuration deployed to the target tenant. Users need access to validate subsequent deployments, and IAM policies reference resources that will be

created in later steps — but the group structure and base policies can be deployed now.

SAML/SSO Configuration

Step	Action	Notes
1	Navigate to **Account Management → Identity & Access Management → SSO**	Target tenant
2	Configure SAML identity provider (same IdP as source if consolidation)	Copy metadata URL from IdP
3	Map IdP groups to Dynatrace groups	Use the IAM design from Step 3
4	Enable SSO enforcement (after initial admin access is confirmed)	Do not lock out admin accounts
5	Test SSO login with at least two different group memberships	Verify policy inheritance

> **Warning:** Do not enable SSO enforcement until you have confirmed that at least one admin account can log in via SSO. A misconfigured SSO with enforcement enabled will lock all users out of the tenant.

Terraform IAM Deployment

IAM is **Terraform-only** — Monaco cannot manage account-level IAM. Deploy the group structure and base policies designed in Step 3:

```
# Initialize Terraform with Dynatrace provider
cd terraform/iam
terraform init

# Plan IAM deployment
terraform plan -var-file="target-tenant.tfvars"

# Apply IAM configuration
terraform apply -var-file="target-tenant.tfvars"
```

IAM Deployment Order

Order	Resource	Terraform Resource Type
1	Groups	`dynatrace_iam_group`
2	Policies	`dynatrace_iam_policy`
3	Policy bindings	`dynatrace_iam_policy_bindings`

> **Note:** For the initial deployment, create policies with broad `ALLOW` statements. After all configuration is imported (Step 5) and validated, tighten policies with `WHERE` clauses that reference specific buckets, schemas, and security contexts.

3. Monaco Bulk Export

Monaco `download` exports all configuration from the source tenant into a structured directory. This export becomes the input for `monaco deploy` in Step 5, or can be packaged for upload to the SaaS Upgrade Assistant (SUA) on the target tenant.

Export Commands

```
# Set environment variables
export DT_SOURCE_URL="https://&lt;source-env-id&gt;.live.dynatrace.com"
export DT_SOURCE_TOKEN="dt0c01.xxx..."

# Full export of source tenant
monaco download \
  --environment-url "$DT_SOURCE_URL" \
  --token "$DT_SOURCE_TOKEN" \
  --output-folder ./export \
  --force

# Export specific configuration types only
monaco download \
  --environment-url "$DT_SOURCE_URL" \
  --token "$DT_SOURCE_TOKEN" \
  --output-folder ./export-settings \
  --only-settings \
  --force
```

Automated Export with SUA Packaging

For a streamlined workflow that handles Monaco download, checksum verification, and SUA-compatible `.tar.gz` packaging in a single command, see **S2S-10: Migration Scripts**. The scripts automate:

- Platform-aware Monaco binary download (macOS, Linux, Windows)
- Temporary read-only API token creation with minimal scopes
- Full `monaco download` execution
- Packaging in SaaS Upgrade Assistant format (`.tar.gz` with `exportMetadata.json`)

> **Important:** The SaaS Upgrade Assistant requires `.tar.gz` format — `.zip` archives are rejected. If packaging manually, ensure the archive contains an `exportMetadata.json` at the root and the exported config in an `export/` subdirectory. See **S2S-10** for the exact format specification.

Export Directory Structure

After `monaco download`, the export directory contains:

```
export/
├── project/
│   ├── alerting-profile/          # Alerting profiles
│   ├── anomaly-detection-*/       # Anomaly detection rules
│   ├── auto-tag/                  # Auto-tag rules
│   ├── dashboard/                 # Classic dashboards
│   ├── management-zone/           # Management zones
│   ├── notification/              # Notification integrations
│   ├── request-attributes/        # Request attributes
│   ├── slo/                       # SLO definitions
│   ├── synthetic-*/               # Synthetic monitors
│   └── ... (50+ config types)
├── manifest.yaml              # Monaco manifest
└── delete.yaml                # Deletion tracking
```

Export Validation

After export, validate the output before proceeding:

```
# Count exported configuration items
find ./export/project -name "*.json" -o -name "*.yaml" | wc -l

# Verify manifest is valid
monaco deploy --dry-run \
  --environment-url "$DT_TARGET_URL" \
  --token "$DT_TARGET_TOKEN" \
  --manifest ./export/manifest.yaml

# Check for entity ID references that need remapping
grep -r "HOST-\|SERVICE-\|PROCESS_GROUP-" ./export/project/ | wc -l
```

> **Important:** Monaco download **cannot export** the following:
> - Cloud provider credentials (AWS IAM roles, Azure service principals, GCP service account keys)
> - Account-level IAM (groups, policies, bindings) — use Terraform
> - ActiveGate tokens and connection info
> - Synthetic private location credentials
>
> These must be recreated manually in the target tenant.

4. ActiveGate Provisioning

ActiveGates in the source tenant cannot be "moved" to the target tenant. New ActiveGates must be deployed and connected to the target tenant. The source ActiveGates remain operational until cutover is validated.

ActiveGate Sizing

Role	CPU	Memory	Disk	Notes
Environment ActiveGate	2+ cores	4+ GB	20 GB	Routing, API access
Cluster ActiveGate	4+ cores	8+ GB	50 GB	Extensions, synthetic
Multi-purpose	4+ cores	8+ GB	50 GB	Combined routing + extensions

Deployment Steps

Step	Action	Validation
1	Download ActiveGate installer from target tenant	Verify installer matches target tenant ID
2	Deploy to designated hosts (same hosts or new hosts)	Check `oneagentctl --get-server` output
3	Assign ActiveGate groups (zones) matching source topology	Verify group assignment in target tenant UI
4	Configure network zones if used	Match source network zone structure
5	Verify connectivity from monitored hosts to new AGs	Test TCP connectivity on port 443

ActiveGate Zone Assignment

If the source tenant uses ActiveGate groups (zones) to segment traffic, replicate the same zone structure in the target tenant:

Source Zone	Purpose	Target Zone	ActiveGate Count
`us-east-prod`	Production workloads	`us-east-prod`	2 (HA pair)
`us-east-nonprod`	Dev/staging	`us-east-nonprod`	1
`eu-west-prod`	EU production	`eu-west-prod`	2 (HA pair)
`extensions`	Extension execution	`extensions`	1 (host-based only)

> **Extensions 2.0 requirement:** Extensions 2.0 require a **host-based ActiveGate** — not a Kubernetes-based ActiveGate. Plan at least one host-based AG if extensions are in scope.

Verify ActiveGate Connectivity

After deploying ActiveGates to the target tenant, verify they are reporting:

```
// Target tenant: verify ActiveGates are connected and reporting
// Note: ActiveGate entity type varies by deployment; host entities report AG
status
fetch dt.entity.host
| filter contains(entity.name, "activegate") or contains(entity.name,
"ActiveGate")
| fields entity.name, id, state
| sort entity.name asc
```

5. Kubernetes Operator Preparation

For environments running Kubernetes, the Dynatrace Operator must be configured to report to the target tenant. This involves updating the DynaKube custom resource and Helm values.

Preparation Steps

Step	Action	Notes
1	Generate new API token and PaaS token in target tenant	Scopes: `activeGateTokenManagement.create`, `entities.read`, `DataExport`, `metrics.read`
2	Create Kubernetes secret for target tenant	`kubectl create secret generic dynakube-target --from-literal=apiToken=<token> --from-literal=dataIngestToken=<token>`
3	Prepare updated DynaKube CR with target tenant URL	Do **not** apply yet — wait for Step 5
4	Validate Helm chart version compatibility	Target tenant cluster version must support the operator version

DynaKube CR Template (Target Tenant)

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://&lt;target-env-id&gt;.live.dynatrace.com/api
  tokens: dynakube-target
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          operator: Exists
  activeGate:
    capabilities:
      - routing
      - kubernetes-monitoring
      - dynatrace-api
    resources:
      requests:
        cpu: 500m
        memory: 512Mi
      limits:
        cpu: "1"
        memory: 1Gi
```

Helm Chart Preparation

```
# Add Dynatrace Helm repo (if not already added)
helm repo add dynatrace
https://raw.githubusercontent.com/Dynatrace/dynatrace-
operator/main/config/helm/repos/stable
helm repo update

# Check available versions
helm search repo dynatrace/dynatrace-operator --versions

# Dry-run to validate (do NOT install yet)
helm upgrade --install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --create-namespace \
  --dry-run
```

> **Do not apply the DynaKube CR yet.** Applying it now would cause agents to start reporting to the target tenant before configuration is imported. Wait for Step 5.

6. Configuration Freeze

A configuration freeze prevents drift between the exported configuration and the source tenant during the migration window. Changes made after the export but before import will be lost.

Freeze Scope

Category	Frozen?	Notes
Settings changes	Yes	No new alerting rules, detection rules, or dashboards
Management zone changes	Yes	No new zones or rule modifications
Auto-tag rule changes	Yes	No new tags or rule modifications
SLO changes	Yes	No new SLOs or metric expression changes
OneAgent deployments	No	New agent deployments continue (they auto-register)
Application deployments	No	Normal CI/CD continues
detected event generation	No	Monitoring remains fully active

Freeze Communication Template

Send to all Dynatrace users and stakeholders:

```
SUBJECT: Dynatrace Configuration Freeze — [DATE] to [DATE]

A configuration freeze is in effect for the Dynatrace source tenant
from [START DATE/TIME] to [END DATE/TIME].

During this period:
- Do NOT create or modify dashboards, SLOs, alerting rules, or settings
- Do NOT create or modify management zones or auto-tag rules
- Normal application deployments and monitoring continue as usual

Reason: SaaS-to-SaaS migration in progress. Configuration will be
imported to the target tenant and changes made after the export
snapshot will not be migrated.

Contact: [Migration Lead] for exceptions.
```

Monitoring for Freeze Violations

Use the audit log to detect any configuration changes made during the freeze window:

```
// Source tenant: detect configuration changes during freeze window
// Run this periodically during the freeze to catch violations
fetch logs, from:-24h
| filter matchesPhrase(log.source, "audit")
| filter contains(content, "CREATE") or contains(content, "UPDATE") or
contains(content, "DELETE")
| filterOut contains(content, "token")  // Exclude token operations
| summarize changes = count(), by:{dt.audit.user}
| sort changes desc
| limit 20
```

7. Rollback Procedure

Every migration must have a documented rollback procedure. Rollback is possible at any point during Steps 4–6 because the source tenant remains fully operational.

Rollback Decision Criteria

Signal	Severity	Rollback?
Entity count < 80% of baseline	Critical	Yes — agents not connecting

Signal	Severity	Rollback?
No log data in target after 1 hour	Critical	Investigate first, rollback if unresolved in 2 hours
detected problems in target tenant unrelated to migration	Low	No — expected during transition
Dashboard shows no data	Medium	No — likely entity ID remapping issue (fixable)
IAM users cannot log in	Critical	Yes — SSO misconfiguration

Rollback Steps

Step	Action	Time Estimate
1	Revert OneAgent `--set-server` to source tenant URL	15 min per wave
2	Revert DynaKube CR `apiUrl` to source tenant	5 min
3	Verify agents reconnect to source tenant	15 min
4	Lift configuration freeze on source tenant	Immediate
5	Notify stakeholders	Immediate
6	Post-mortem and re-plan	Next business day

> **Key principle:** The source tenant is never decommissioned until the migration is fully validated (Step 9). Rollback is always possible by pointing agents back to the source tenant.

8. Step Completion Checklist

Before proceeding to Step 5 (Execute), verify all preparation tasks are complete:

Deliverable	Status	Owner	Notes
Target tenant provisioned	☐	Platform	Environment active, Grail enabled

Deliverable	Status	Owner	Notes
SSO configured and tested	☐	Security / Platform	At least two group memberships validated
IAM deployed via Terraform	☐	Platform	Groups, policies, bindings applied
Monaco export completed	☐	Platform	Full export validated with dry-run
Entity ID audit completed	☐	Platform	Hardcoded IDs identified and remapping plan ready
ActiveGates deployed	☐	Platform / Infra	All zones covered, connectivity validated
K8s operator prepared	☐	Platform / K8s	DynaKube CR ready, secrets created, Helm validated
Configuration freeze active	☐	All	Freeze communicated, monitoring for violations active
Rollback procedure documented	☐	Platform	Reviewed and approved by migration lead
Pre-migration baseline recorded	☐	Platform	Entity counts, bucket inventory captured

> **Gate check:** Do not proceed to Step 5 until all items are checked. Missing preparation creates compounding problems during execution.

Next Step

Continue to **S2S-05: Step 5 — Execute: Configuration Import and Agent Cutover** to import configuration and redirect agents to the target tenant.

Completed	Next	Remaining
~~1. Discover~~ → ~~2. Strategize~~ → ~~3. Design~~ → ~~4. Prepare~~	**5. Execute**	6. Integrate → 7. Expand → 8. Enable → 9. Optimize

---